

Module: TM470

Assessment: EMA



Andrew Forrest

OUCU - zu514945

Date: 16th September 2024

Table of Contents

1. Preparation and Planning	4
1.1. Project Title.....	4
1.2. Project Description and Scope.....	4
1.2.1. The Problem.....	4
1.2.2. The Solution.....	4
1.2.3. Key ICT Aspects of the Solution	5
1.2.4. Skills Required.....	6
1.2.5. Existing Knowledge of the Problem	6
1.2.6. Risks to the Project	7
1.3. Resources.....	9
1.3.1. Resources Needed.....	9
1.3.2. Availability of Resources.....	9
1.3.3. Achievable Estimate	9
1.4. Future Plan.....	10
2. Project Work Completed	11
2.1. Information Sources	11
2.2. Project Work.....	13
2.2.1. FastAPI	13
2.2.2. Wireframes	14
2.2.3. Frontend.....	14
3. Review and Reflection.....	16
3.1. Review of Current Stage of Project Work	16
3.2. Legal, Social, Ethical and Issues.....	16
3.3. Professional Issues	18
3.4. Equality, Diversity and Inclusion Concerns.....	18
3.5. Review Project Management.....	19
3.6. Review of Personal Development	19
Bibliography	20
Appendix.....	23
1. Screenshot starting Devstack	23
2. Screenshot of FastAPI OpenStack Route	23
3. Screenshot of AWS Request using Postman.....	24
4. Screenshot of AWS FastAPI Routes	25
5. Screenshot of application getting registered with Azure.....	26
6. Screenshot of Azure request using Postman.....	27
7. Screenshot wireframe of user that hasn't connected their AWS account	28
8. Screenshot wireframe of AWS servers page.....	29

9. Screenshot of CORS implementation.....	30
10. Screenshot of AzureVM page	30
11. AzureVM Component Code	30
11. Sidebar Component Code.....	34
13. Screenshot of Azure Containers Page.....	38
14. AzureContainers Component Code	38
15. Screenshot of OpenStack Servers page.....	42
16. OpenStackServers Component Code.....	42
17. Screenshot of AWSVolumes Page	46
18. AWSVolumes Component Code	46
20. AWSEC2Instances Component Code.....	49
21. Screenshot of S3Buckets Page	53
22. S3Buckets Component Code.....	53

1. Preparation and Planning

1.1. Project Title

Development of a Multi-Cloud Management Dashboard

1.2. Project Description and Scope

1.2.1. The Problem

According to Forbes, 98% of enterprises plan to or are currently using a multi-cloud strategy in their development (Fay, 2023). To use the services provided by different vendors, enterprises will either have to use multiple different dashboards from each of the cloud vendors or use an infrastructure as code tool such as terraform (Morris, 2016).

The problem with this is the complexity and management overhead associated with deploying and managing infrastructure across multiple cloud service providers. This is caused by having to deal issues such as differing APIs, abstractions and security challenges (Saxena et al., 2021).

From research carried out, I was only able to find one company who provides a multi-cloud management dashboard solution - mist.io, however they have been acquired by Dell and have dropped support for their community edition, therefore forcing users to pay for their proprietary offering (Mist.io, 2023).

Therefore, there appears to not be any non-proprietary software available that allows users to manage their multi-cloud infrastructure using their own self hosted dashboard.

1.2.2. The Solution

I would like to create a self-hostable multi-cloud dashboard to help enterprise users provision and manage their cloud resources.

The system provided will allow users to connect their cloud accounts to the system, which once done, the management dashboard will be configured to provision and manage their own cloud resources such as virtual machines, security groups, networks, load balancers and storage. As well as be able to monitor and view logs on aspects of their cloud infrastructure such as instance uptime and access logs.

The benefit of this solution is that users that prefer development using a graphical user interface can manage their multi-cloud deployment infrastructure on a single unified user interface, as opposed to using multiple cloud vendor user interfaces, or have to use an infrastructure as code tool.

1.2.3. Key ICT Aspects of the Solution

Key aspects in completing the project include:

1. Backend Development with the Python programming language and the FastAPI framework. The system will leverage the FastAPI framework – a web framework designed for developing APIs using python, which has built in functionality such as type declarations and self-documentation (FastAPI, 2024). Python is a programming language with a simple syntax that is well suited for rapid development environments (Drake, 2003, p. 115). These technologies will be used to work alongside cloud vendor SDKs to handle the core business logic of the system, such as provisioning compute infrastructure on AWS, then exposing this as an API to be consumed by the frontend dashboard. An SDK is a group of libraries, tools, documentation, processes and or guides that allow developers to create applications for a specific platform (Sandoval, 2016).
2. Frontend Development with the Typescript programming language, HTML, CSS and the React library. Using the frontend stack above will be used to develop the dashboard user interface for the system. Typescript is a strongly typed programming language that is built on top of JavaScript which can help find errors in code before being run in production (typescriptlang, 2020). HTML is a markup language that defines the structure of a web page (W3schools.com, 2024). CSS is a language used to define how each HTML element should be displayed (W3schools.com, 2024).
3. User Experience and Design. Consideration will be needed to allow ease of use and adhere to accessibility legislation such as the Equality Act 2010. Examples of this would be ensuring the website design has a suitable contrast ratio between colours and reducing the distance the user must travel to get to a particular resource (Hassenzahl & Tractinsky, 2006), this would help ensure compliance with legislation and improve user satisfaction and engagement.
4. Authentication and authorization. It will be essential to properly manage private confidential data that will be getting sent over the internet. This will therefore require appropriate data security practices such as implementing encryption by for instance leveraging HTTPS instead of HTTP. This would help reduce the risk of man in the middle attacks which is where a malicious actor is able to intercept and read communication messages between two or more systems or individuals (Mallik, 2019).
5. Being able to work with Restful APIs will also be core to this project. A restful API is a way of connecting and integrating with a software application to help build a distributed system (Biehl, 2016). Examples of using an API for this project include having the frontend React application interact with the FastAPI backend, as well as have the FastAPI backend interact with cloud vendor SDKs which will interact with the cloud vendors available APIs.

1.2.4. Skills Required

To successfully complete this project, I will need to have a good understanding of both frontend and backend development, as well as have a good understanding of cloud technology.

With regards to frontend development, I will need to know how to leverage the React library effectively, where I will need to understand how to consume and render the relevant API data sent from the API to the user through each component I develop.

Regarding the backend, I will need to understand how to create and develop an API, where skills knowing how to send the correct HTTP responses, such as 404 if a resource can't be found or 200 if the response is successful. I will also need to have skills in understanding handling secure data within the backend, such as knowing how to handle environment variables which can be used to manage users' cloud credentials.

With cloud technologies, I will need to have skills in how to use the different SDKs provided by each of the cloud providers, and how to interact with each of their services on behalf of the user. I will also need to do this, such that user security isn't compromised.

1.2.5. Existing Knowledge of the Problem

From working with on my university module: TM352 Web, mobile and cloud technologies. I have had to work with the cloud vendors AWS, OpenStack and Azure as part of my coursework. Examples of work carried out using these technologies include researching and developing cloud architectures to meet the requirements for an application for a client. This meant I had to learn about different solutions offered by each of these vendors to create a scalable architecture using core components such as a web server, load balancer and database.

In my TM252 module I have also had extensive use of the React library and the FastAPI framework as part of coursework. For instance, in one of my assignments I had to develop a toll payment application, this had the frontend consume the FastAPI backend data and then render the correct data to the user, then successfully send the user input data to the backend for validation to successfully book and pay for their toll.

I have also got a lot of experience working with AWS for my own personal projects, such as hosting web applications using the Laravel framework. In addition to this, last year I wanted to further deepen my knowledge of AWS and went and passed an official AWS certification offered by AWS – AWS Certified Developer Associate. To achieve this certification, I had to display excellent understanding of best practices of a large array of services offered by AWS such as Lambda, RDS, SQS, IAM and handle CI/CD pipelines.

From my experiences from coursework and personal projects I feel I have a good knowledge base to draw from to help deliver the proposed solution. As many of the services and

technologies I will need to use to complete the solution, will be services I have already used or have a high-level understanding of how they should be used.

1.2.6. Risks to the Project

1. Unable to complete all features of the project solution.

Due to the number of features to be added to the project, it is possible that I am not able to complete all of them before the end of module. This would likely occur because of an unexpected issue occurring within the development of the project, such as an issue authenticating against a particular service that I haven't used before. Although this isn't impossible there is a medium risk of this occurring. However, to mitigate this, I have allotted myself additional time in my project schedule to handle any unforeseen circumstances.

2. User credentials get leaked.

It is possible that user credentials get leaked if the application has any bugs when handling users' private data. This could occur by for instance not storing and submitting these credentials securely. Though it is not impossible for mistakes to be made when working with user data, I will ensure that all work carried out adheres to the recommended security practices advised by each of the cloud vendors as well as from the official FastAPI and React documentation.

3. The application doesn't work as expected on their machine as it does on mine

It is possible that I can have my application working exactly as intended on my machine with it successfully interacting with the cloud services as intended. However, that doesn't guarantee that this will be the case with users of the application. There is a medium risk of this being the case. To mitigate, thorough testing of the application should occur, as well as have a recommended environment that users should configure on their own machine that matches what I have been developing with to help reduce this risk.

4. Cloud vendors modify or deprecate features of their SDK or services that are being used by the application.

It is possible that over the course of developing this project, that one of the cloud services that I have configured the application to work with will make modifications to their services that can cause unexpected behaviour to my application. This is of a medium risk to the project as there is no way for me to know if or when these changes occur. The best course of action is to ensure that extra time in my project schedule is free to give me time to make any needed modifications to the project if any unexpected problems arise.

5. My hardware used to develop the project breaks.

It is possible over the course of development, that my computer breaks whilst working on the project. This would naturally cause significant disruption to the development of the project as I wouldn't be able to carry out any work whilst I don't have a machine to work with. This is of low risk as I have a new machine that I am working with, and I have allocated myself space in my project schedule to deal with any unexpected issues.

1.3. Resources

1.3.1. Resources Needed

To complete the project, resources needed include access to a development environment to deploy the application. This could be in the form of an EC2 instance running Ubuntu, which will allow me to install dependencies such as python, FastAPI, React and SDKs from cloud vendors such as that from AWS, Azure and OpenStack.

As for resources of information to help assist in carrying out the project. Good documentation on how to use technologies such as the Python programming language and use it to interact with cloud services.

1.3.2. Availability of Resources

The main resources used in carrying out development will be through reading through the official documentation from the cloud vendors, programming languages, frameworks and libraries used. This will be used to read up on the various APIs available and ways in which the SDKs can be used. I will also look to tutorials and blog posts to help see how different people have used these technologies in ways which weren't explicitly mentioned in the documentation.

As for personnel providing help if there is difficulty encountered from understanding the documentation or other. There are no individuals who are experts who I could go to. However, there are many public forums that can provide help such as StackOverflow.

1.3.3. Achievable Estimate

Due to time constraints and the complexity of the project, I will aim to be able to provide support for at least two cloud providers but will try to get three done if possible. The cloud vendors I will target to provide initial support for will be AWS, OpenStack and Azure. These are the target vendors as AWS and Azure are the biggest cloud vendors in the world and I am familiar with OpenStack due to coursework completed within my module Web, Mobile and Cloud Technologies.

Due to lack of time on providing support for multiple environments, it may be the case that the developer may only support the software being downloaded and run on a certain environment such as on an EC2 small instance running the Ubuntu operating system.

As for the minimal features that are achievable within the timeframe would be:

- Ability to download and run the software onto the user's machine.
- User can connect their account to the software.
- Dashboard will be able to show resources currently in use from their cloud such as their virtual machines provisioned, objects in storage, networks and volumes in use.
- User will be able to see the region of where their resources are located.
- User will be able to stop, start and terminate their virtual machine images.
- User will be able to upload and delete objects in storage.

- User can add or delete their networks and volumes.
- User can successfully log out of their accounts.

1.4. Future Plan

After completing the minimum requirements of the project, additional features of the system could be expanded. Examples of this include:

- Being able to modify resources once created, such as changing the EC2 instance type from a T2.micro to a m7g.medium.
- Being able to mount volumes to an instance.
- Being able to create and manage keypairs.
- Use additional cloud services such as AWS RDS

2. Project Work Completed

2.1. Information Sources

As this project will rely on connecting to users of the applications AWS, OpenStack and Azure accounts, I wanted to learn more about best practices regarding authentication when connecting an external on premises application to their cloud provider. As I am most familiar with AWS, I first went through their official documentation to find information on how authentication should be done when using their python SDK. From the AWS official documentation, there was an article on handling user credentials when connecting to AWS using the SDK. From this article, it discussed how it is best practice to not use hard coded user credentials into the application, and instead opt for the use of environment variables which can be called into the application connection string as parameters (AWS, 2024). An environment variable is a variable that encapsulates data and won't change its value over the execution of a program (Gilliam, Wolfe, Sherif, & Bishop, 2003).

When researching the official OpenStack documentation for handling user credentials correctly, it recommends using a config.yaml file that users can download from their cloud with their API credentials, which applications can use with the OpenStack SDK to parse as an object to be used for authentication.

When looking at Azure, I came across a manual in their official documentation regarding authentication for on premises applications using their python SDK. The azure manual gave multiple different ways of authentication practices that can be used to allow users management of their Azure resources, the one that I would most likely use involves the user having to register the application with their Azure cloud through the portal, then attaching permissions to their registered application, from there the user can get their credentials which can be used as environment variables that the application can use, similar to how AWS SDK works (Microsoft, 2023).

I have also had to go about researching how to create my own local cloud for OpenStack development and testing for the project. Upon researching for a simple solution I came across Devstack, an OpenStack supported project which provides scripts to provision an OpenStack environment for local development purposes (Openstack.org, 2023). As Devstack makes significant changes to the installed system, it's recommended to run it on a virtual machine (OpenStack, 2023). For virtualisation on my machine, I looked into the software tool multipass, which is an Ubuntu virtual machine orchestration tool that will allow me to initialise and configure my devstack environment to run on Ubuntu on my windows machine (Multipass, 2024).

For the development of my API, I have adhered to following the official guides on the FastAPI documentation. From this documentation I have been able to find out how to properly structure my program as well as how to correctly perform HTTP requests such as GET for my AWS EC2 instances. I have particularly found useful the documentation on

using the APIRouter class for handling routes to files outside of my main.py file (FastAPI, 2014), as this is a feature of fastapi that I have not worked with before.

In addition to this, I have also had to learn about cross origin resource sharing (CORS). This is a HTTP header system that allows a backend system to give permission to a different origin to interact with it (Hossain, 2014). In the case of this application this applies to working between the frontend React application and the backend fastapi service.

With regards to ways of testing my API. From my research I have found a tool called Postman. This tool provides a way for me to connect my API to it, where I can then run tests to ensure that variables such as authorisation validation and response messages are working as intended (Postman, 2024).

For the frontend, I have looked at research on how to effectively design my application. I have looked at research looking at the importance of reducing friction by guiding users to their desired location and by looking at making navigation throughout the page accessible, where consideration to factors such as font size and colour contrast should be looked at (Hassenzahl & Tractinsky, 2006).

When working with react, I have researched the official documentation to ensure that I carry out best practices. Examples of this include:

- Breaking the user interface into a component hierarchy, such as having a sidebar app, sidebar and ec2 component.
 - Learning when to use and not use the useEffect hook, when other more optimal hooks may be better suited such as useEffect.
- (React, 2024).

The react application also needed a way of handling application routing. For instance, I would need a way of being able to direct people to the azure-vm page and the aws-s3 page. From my research I found the library, react router to handle this. I chose this library as from my research this appears to be the most common solution to handle client side routing and I also found that they had very comprehensive documentation to help me learn on how to setup the library and work with it in my application (ReactRouter, 2024).

The frontend application will need to be able to interact with the backend. Therefore, I have had to carry out research on how best for this to occur. From my research I have looked at using the Axios JavaScript library to handle this, instead of the conventional fetch method. Axios is a promise based HTTP client that has a number of benefits such as automatic serialisation into JSON, support for old browsers such as internet explorer and additional security against cross-site forgery (Strazzullo, 2019).

2.2. Project Work

2.2.1. FastAPI

Development work for the project backend has entirely consisted of using the Python framework FastAPI. In the sections below I will discuss how I have used FastAPI to interact with each of the cloud vendors and what features I have built on using my API. In addition to creating routes to cloud vendors, I also had to enable CORS to the frontend application, so that users using their browser could make queries to the backend application (see appendix 9).

2.2.1.1. OpenStack

Before connecting my FastAPI to OpenStack, I first had to download and install a local environment of OpenStack onto my machine before this would be possible. For this I used Devstack, which is a local development environment supported by OpenStack (see appendix 1).

Once a local environment was created, I proceeded to create a connection with my FastAPI backend and my OpenStack system. This was done by downloading the OpenStack SDK, then downloading a clouds.yaml file with my connection credentials from my OpenStack system. Once I had those downloads, I had to create a connection using the SDK.

After creating a connection, I began creating custom API requests to the OpenStack cloud. Examples of the requests completed include, GET servers, GET keypairs, GET flavours, GET regions and POST networks and POST server (See appendix 2).

2.2.1.2. AWS

Once I downloaded the AWS python SDK – Boto3 into my development environment, I then generated new user credentials for my application, I was then able to begin development of my API to interact with AWS services.

I have been able to develop API routes such as GET my AWS buckets, GET EC2 instances, POST bucket and POST EC2 instance (See appendix 3 and 4).

2.2.1.3. Azure

Before development of my API to interact with Azure, I had to first register my application with my Azure cloud (See appendix 5). Once registration was completed, I was able to get credentials which I could use with the application. Following this I downloaded the Azure python SDK into my development environment.

I been able to create routes such as GET and POST containers and servers in my Azure cloud (see appendix 6).

2.2.2. Wireframes

I have also begun preparation for the development of the frontend of the application. I have completed wireframes for pages of the application such as the connect cloud page, view servers page, view buckets and create server. This allows me to have a good template to use which I can look at whilst I develop the frontend application.

When creating the wireframes I have tried to make good consideration to user experience, design and accessibility.

For instance, I have tried to minimise the number of clicks needed for a user to achieve a task. One way this has been done is having each of the core functionalities of the application being displayed on a sidebar, such as servers and buckets. This allows a one click system for the user to reach their destination.

Another example is trying to present the content in a simple and easy navigable way. For instance, on the servers page, there is a table with all servers displayed on a table, with columns including key information for each particular server, such as its name and status. This helps provide the user with all of the key information on each server in a simple format, where if they wish for more information on each server, they can click on the server link on the table to get more thorough logs on things such as its uptime or what key pairs are assigned to it.

2.2.3. Frontend

React was used to develop dynamic, interactive user interfaces with a component-based architecture for the application.

This was done by breaking it into modular and reusable components.

The application was structured into components based on different features and functionalities. Key components included:

- **Sidebar** (see appendix 12): The Sidebar component serves as the main navigation menu for the application, providing links to different sections like VM and Container management. Reacts useState hook is used to manage active states or collapsed sections, ensuring a dynamic user experience. Tailwind CSS is used to handle styling to create a vertical, collapsible sidebar, with media queries ensuring that it remains responsive on

smaller screens. SVG icons are often included alongside text labels for better visual communication, making navigation intuitive to the user.

- **AzureContainers** (see appendix 13 and 14): This component manages Azure Containers by displaying the list of available containers, allowing users to create new ones, and providing the ability to delete existing containers. The state management in this component was handled using React's `useState` and `useEffect` hooks to track the container list, loading states, error handling, and user interactions.
- **OpenStackServers** (see appendix 15 and 16): This component manages OpenStack Servers by displaying the list of available servers, allowing users to create new ones, and providing the ability to delete existing servers. The state management in this component was handled using React's `useState` and `useEffect` hooks to track the container list, loading states, error handling, and user interactions.
- **AWSVolumes** (see appendix 17 and 18): This component manages AWS Volumes by displaying the list of available volumes, allowing users to create new ones, and providing the ability to delete existing volumes. The state management in this component was handled using React's `useState` and `useEffect` hooks to track the container list, loading states, error handling, and user interactions.

The Axios library was used within several of my components to allow for user interaction with the fastapi backend. This would be in the form of a GET or POST request to the backend target route. It was important to have appropriate error handling to ensure that any issues such as an invalid container name being used to create an S3 bucket, or a service not being available get displayed to the user to help improve user satisfaction with the system.

Tailwind CSS was used as the main styling framework for its utility-first approach, which helped speed UI development without writing custom CSS. Tailwind CSS was used extensively throughout the application to style components, ensuring a clean and responsive layout. Common utility classes were used to define spacing, typography, colours, and layout structure.

3. Review and Reflection

3.1. Review of Current Stage of Project Work

Overall, I am content with the development progress made with my project. I have successfully been able to connect my backend to all my cloud, made significant progress on the development of the APIs to manage infrastructure from each vendor and developed the frontend application to have a user interface to interact with all APIs built so far that is intuitive and adheres to accessibility legislation. For instance, with AWS I have routes allowing for the creation and deletion of servers, volumes and S3 buckets. With Azure I have routes allowing for the creation and deletion of servers and containers and with OpenStack I have routes allowing for the creation and deletion of servers.

I would however have liked to have more API routes completed with a frontend page to complement it. For instance I would have liked to have had a networks page for all cloud vendors. I would also have liked to have more functionality with the application routes. For instance, with creating an EC2 instance there is only the option to create a T2.micro instance with the default operating system. I would have liked to have the user have control over these variables. There could also be a cleaner way of allowing users to add in their credentials into the system. Currently users will have to manually add their credentials into a .env file in the backend for Azure and AWS, and with Openstack they need their clouds.yaml file. It would have been nice if these credentials could be uploaded securely via the frontend of the application.

If I were to complete the project again, I would try to have better time management of the project to stop me being put under pressure with the deadline. As I feel like I put myself in catchup mode throughout. Such that I spent too much time trying to clean up the application code, and not giving myself enough time on writing my report. This has meant that I have less time to add as much thorough details as possible in my report.

3.2. Legal, Social, Ethical and Issues

1. What is the point of solving the problem?

By creating a multi-cloud management dashboard, users will be able to manage their cloud resources through one unified user interface instead of using multiple differing dashboards from each cloud vendor. Provided no security issues immerge from its use, there will not likely be any negative social impacts.

2. Who will use the results of the project?

Enterprise developers who're responsible for managing their cloud resources.

3. Who will be affected when I am solving the problem?

There will not likely be anyone affected when developing the solution. The only potential stakeholder that could emerge would be if I had an issue that I couldn't solve, I may use a developer forum such as StackOverflow and ask users help with my issue.

4. What could happen if the outcome of the project becomes extremely successful?

Cloud vendors such as AWS may find that user traffic to their own dashboards diminishes whilst demand for their services remain.

5. Could private data be collected?

Yes, user's cloud login credentials will be needed in order to connect the application with their cloud resources.

6. Could people be tracked?

Yes, users could be tracked on metrics such as what infrastructure they are using in their cloud systems.

7. Could the outcome of the project be misused?

Yes, it could be possible to steal users sensitive cloud credentials and then use these to hack their infrastructure.

8. If the project has any unexpected errors in logic or security holes, could this have any bad impacts?

Yes, users of the system could end up getting their cloud infrastructure hacked if their user credentials were compromised.

9. Are there any relating to data protection, computer misuse or freedom of information?

Yes, the application will have to adhere to GDPR legislation.

10. Does my project contribute towards a larger development in technology that leads somewhere you feel may or may not be for the good of society?

No, this project is only a tool that's used to make it more convenient to use technology that is already available.

3.3. Professional Issues

1. Do I have any conflicts of interest?

No, I don't have any conflicts of interest to develop this project.

2. Are the project outcomes fit for purpose?

Yes, the outcomes of this project are fit for any potential client who would want a tool to help manage their multi-cloud infrastructure.

3. Will I be using appropriate tools and techniques to complete the project?

Yes, I will be using the latest official SDKs for AWS, Azure and Openstack. I will also be working with the latest long term supported version of python and React to complete the project.

I will also ensure that I adhere to the best practices that are advised for all software that I will be using.

4. Will I be using pirated or software I haven't permission to use?

No.

5. Do I have the necessary skills to complete the project?

As outlined in 1.2.4. I should have adequate skills to complete the project.

6. How will I take responsibility if the project doesn't work out as planned?

If I am unable to complete the project to an appropriate level of completeness, I will not release the software publicly to avoid risk to potential users being exposed to any security vulnerabilities as a result of bugs within the code.

3.4. Equality, Diversity and Inclusion Concerns

Due to the nature of the project, there should not be any challenges in ensuring that this project has no bias against any equality or diversity concerns. However, due to the interactivity needed for users to use the solution, there is a concern that without proper planning for accessibility considerations for this project, then this may not be inclusive. Examples of considerations to ensure that the project is accessible include making sure that the user interface can be navigated with the use of a screen reader and ensuring that there is an appropriate contrast between colours.

3.5. Review Project Management

The chosen lifecycle for the project was the scrum framework.

When reflecting on my most recent work, I would say that I haven't strictly been following the scrum framework as well as I would have liked. I have most recently been blocking off free study time to focus on one large area at a time and continually working on that task until finished. For instance, with working on OpenStack, I would just work on that work for the week until I complete what was needed to be done in the schedule.

I do however think that I should make a better effort at working with this methodology for my project, as when I have been more honest with myself when adhering to scrum I have felt as though I have been able to achieve the tasks I have set for myself. Where each week I set new sprints to complete for each week. Using this lifecycle model has made me help plan for the short and long term by having me evaluate how my work is going each week and then adjusting each sprint to ensure that I can keep on target,

3.6. Review of Personal Development

The work completed so far in this project has been related to working with FastAPI with cloud SDKs and React. I have greatly improved my proficiency with these technologies over the course of development. For instance, I have never had to work with the OpenStack SDK before stating this project. I have also never had to work with the axios library when working with APIs before. From working on this project, I have been able to experience working with these new technologies and learn how to debug tools which I am not familiar with. This has meant having to learn how to quickly find key relevant sources of information that can help me solve a particular problem as well as learn how to structure my time more effectively to manage new challenges.

I do still think that I can improve personally with better time management, where I would be able to keep a better routine with my work. This would help me not be in any rush to help find solutions to any problems I encounter, which would alleviate the probability of making any silly mistakes, such as writing the wrong variable names in my code which leads to further bugs.

Bibliography

AWS. (2024). Available at:

https://docs.aws.amazon.com/pythonsdk/?icmpid=docs_homepage_devtools [Accessed 16 Apr. 2024].

AWS. (2024). *A Sample Tutorial - Boto3 1.34.84 documentation*. [online] Available at:

<https://boto3.amazonaws.com/v1/documentation/api/latest/guide/sqs.html> [Accessed 16 Apr. 2024].

Biehl, M., 2016. *RESTful Api Design* (Vol. 3). API-University Press.

Drake, F.L. ed., 2003. *An introduction to Python* (p. 115). Bristol: Network Theory Ltd..

FastAPI. (2014). *APIRouter class - FastAPI*. [online] Available at:

<https://fastapi.tiangolo.com/reference/apirouter/?h=apirouter> [Accessed 18 May 2024].

FastAPI. (2024). *FastAPI*. [online] Available at: <https://fastapi.tiangolo.com/> [Accessed 15 Jul. 2024].

Fay, G. (2023). Council Post: Will Multi-Cloud Environments Ever Be Truly Secure? *Forbes*. [online] 30 Jun. Available at: <https://www.forbes.com/sites/forbestechcouncil/2023/06/29/will-multi-cloud-environments-ever-be-truly-secure/> [Accessed 15 Apr. 2024].

Gilliam, D.P., Wolfe, T.L., Sherif, J.S. and Bishop, M., 2003, June. Software security checklist for the software life cycle. In *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003*. (pp. 243-248). IEEE.

Hassenzahl, M., & Tractinsky, N. (2006). User experience – a research agenda. *Behaviour & Information Technology*, 25(2), 91-97.

Hossain, M., 2014. *CORS in Action: Creating and consuming cross-origin apis*. Simon and Schuster.

Mallik, A., 2019. Man-in-the-middle-attack: Understanding in simple words. *Cyberspace: Jurnal Pendidikan Teknologi Informatika*, 2(2), pp.109-134.

Microsoft. (2023). *Authenticate to Azure resources from Python apps hosted on-premises - Python on Azure*. [online] Microsoft.com. Available at: <https://learn.microsoft.com/en-us/azure/developer/python/sdk/authentication-on-premises-apps?tabs=azure-portal> [Accessed 18 May 2024].

Mist.io. (2023). *Thank You for Being Part of Our Journey*. [online] Available at: <https://mist.io/blog> [Accessed 15 Apr. 2024].

Morris, K., 2016. *Infrastructure as code: managing servers in the cloud*. " O'Reilly Media, Inc."

Multipass. (2024). *Multipass orchestrates virtual Ubuntu instances*. [online] Available at: <https://multipass.run/> [Accessed 18 May 2024].

Openstack.org. (2021). *Getting started — openstacksdk 3.1.0.dev15 documentation*. [online] Available at: <https://docs.openstack.org/openstacksdk/latest/user/guides/intro.html> [Accessed 16 Apr. 2024].

Openstack.org. (2023). *DevStack — DevStack documentation*. [online] Available at: <https://docs.openstack.org/devstack/latest/> [Accessed 18 May 2024].

Openstack.org. (2023). *Using the OpenStack SDK — openstacksdk 3.1.0.dev15 documentation*. [online] Available at: <https://docs.openstack.org/openstacksdk/latest/user/index.html> [Accessed 16 Apr. 2024].

React. (2024). *You Might Not Need an Effect – React*. [online] Available at: <https://react.dev/learn/you-might-not-need-an-effect> [Accessed 16 Sep. 2024].

React Router. (2014). *Feature Overview v6.26.2 | React Router*. [online] Available at: <https://reactrouter.com/en/main/start/overview> [Accessed 16 Sep. 2024].

Sandoval, K. (2016). *What is the Difference Between an API and an SDK? | Nordic APIs* |. [online] Nordic APIs. Available at: <https://nordicapis.com/what-is-the-difference-between-an-api-and-an-sdk/> [Accessed 15 Jul. 2024].

Saxena, D., Gupta, R. and Singh, A.K., 2021. A survey and comparative study on multi-cloud architectures: emerging issues and challenges for cloud federation. *arXiv preprint arXiv:2108.12831*.

Srivastava, A., Bhardwaj, S. and Saraswat, S., 2017, May. SCRUM model for agile methodology. In *2017 International Conference on Computing, Communication and Automation (ICCCA)* (pp. 864-869). IEEE.

Strazzullo, F. (2019) 'HTTP Requests', in *Frameworkless Front-End Development*. United States: Apress L. P, pp. 113–138. Available at: https://doi.org/10.1007/978-1-4842-4967-3_5.

Typescriptlang. (2020). *JavaScript With Syntax For Types*. [online] Available at: <https://www.typescriptlang.org/> [Accessed 15 Jul. 2024].

W3schools.com. (2024). *W3Schools.com*. [online] Available at: <https://www.w3schools.com/Css/> [Accessed 15 Jul. 2024].

W3schools.com. (2024). *W3Schools.com*. [online] Available at: <https://www.w3schools.com/html/> [Accessed 15 Jul. 2024].

Appendix

1. Screenshot starting Devstack

```
stack@devstack:~$ cd devstack/
stack@devstack:~/devstack$ ls
CONTRIBUTING.rst  LICENSE      clean.sh    extras.d    functions-common  lib      playbooks  samples  tests  unstack.sh
FUTURE.rst        Makefile    data       files      gate            local.conf  roles      stack.sh  tools
HACKING.rst       README.rst  doc       functions  inc            openrc     run_tests.sh  stackrc  tox.ini
stack@devstack:~/devstack$ ./stack.sh
```

2. Screenshot of FastAPI OpenStack Route

```
1  import openstack
2  from dotenv import load_dotenv
3  from fastapi import APIRouter
4
5  router = APIRouter(prefix="/openstack", tags=["OpenStack"])
6
7  load_dotenv()
8
9
10 conn = openstack.connect(cloud="openstack")
11
12
13 @router.get("/servers")
14 def list_servers():
15     print("List Servers:")
16     for server in conn.compute.servers():
17         return {"server": server.to_dict()}
18
```

3. Screenshot of AWS Request using Postman

The screenshot shows the Postman interface for a GET request to `127.0.0.1:8000/aws/buckets`. The request is saved and ready to be sent. The response is a 200 OK status with a response time of 869 ms and a body size of 254 B. The response body is displayed in JSON format, showing a list of bucket names under the key "buckets".

Request Details:

- Method: GET
- URL: `127.0.0.1:8000/aws/buckets`

Response Details:

- Status: 200 OK
- Time: 869 ms
- Size: 254 B

Response Body (JSON):

```
{
  "buckets": [
    "andrewforrest.me",
    "apple-data-tarball-encrypted",
    "get-a-car",
    "laravel-10-s3-demo",
    "windows-bucket-archive-dec-22"
  ]
}
```


4. Screenshot of AWS FastAPI Routes

```
6  router = APIRouter(prefix="/aws", tags=["AWS"])
7
8  load_dotenv()
9  ACCESS_KEY = os.getenv("aws_access_key_id")
10 SECRET_KEY = os.getenv("aws_secret_access_key")
11
12
13 def get_session():
14     session = boto3.Session(
15         aws_access_key_id=ACCESS_KEY, aws_secret_access_key=SECRET_KEY
16     )
17     return session
18
19
20 @router.get("/buckets/")
21 def list_buckets():
22     session = get_session()
23     s3_client = session.client("s3")
24     response = s3_client.list_buckets()
25     buckets = [bucket["Name"] for bucket in response["Buckets"]]
26     return {"buckets": buckets}
27
28
29 @router.get("/instances")
30 def list_ec2_instances():
31     region = os.getenv("aws_region")
32     session = get_session()
33     ec2_client = session.client("ec2", region_name=region)
34     response = ec2_client.describe_instances()
35     return {"ec2 instances": response}
36
```

5. Screenshot of application getting registered with Azure

Microsoft Azure

Search resources, services, and docs (G+/)

forrestandrew7@gmail...
DEFAULT DIRECTORY

Home > App registrations >

Register an application

* Name

The user-facing display name for this application (this can be changed later).

multicloud-management-dashboard

Supported account types

Who can use this application or access this API?

☒ Accounts in this organizational directory only (Default Directory only - Single tenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)

Redirect URI (optional)

By proceeding, you agree to the Microsoft Platform Policies

Register

6. Screenshot of Azure request using Postman

The screenshot shows the Postman interface for a GET request. The URL bar displays '127.0.0.1:8000/azure/containers'. The 'Send' button is visible. Below the URL bar, the 'Params' tab is active, showing a table for Query Params. The response body is displayed in the 'Body' tab, showing a JSON object with a 'Containers' property set to 'demoimagecontainer'.

HTTP multicloud-management-dashboard / **Get Azure Containers** Save Share

GET 127.0.0.1:8000/azure/containers Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (4) Test Results 200 OK 1208 ms 160 B Save as example

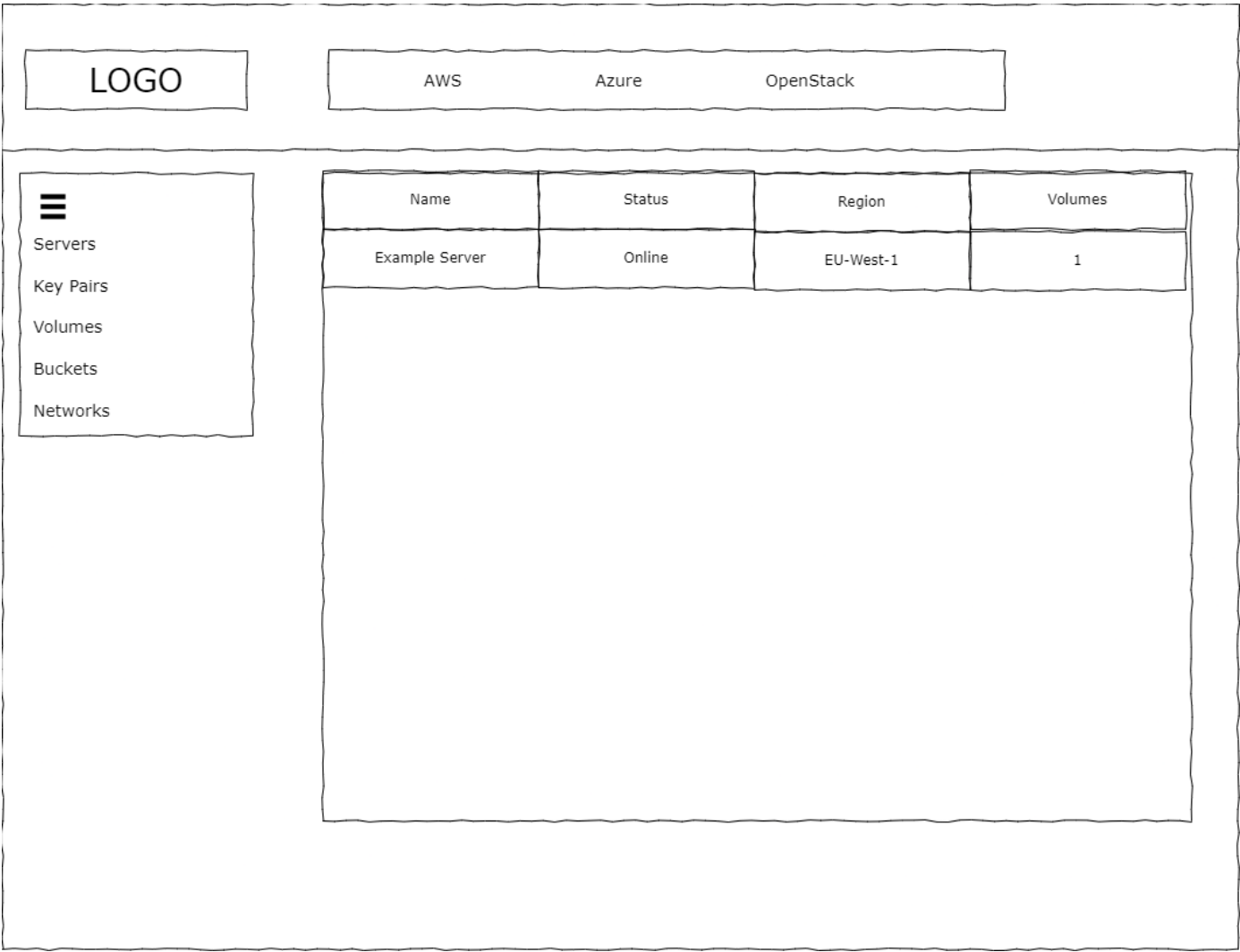
Pretty Raw Preview Visualize **JSON** Copy Search

```
1 {  
2   "Containers": "demoimagecontainer"  
3 }
```

7. Screenshot wireframe of user that hasn't connected their AWS account



8. Screenshot wireframe of AWS servers page

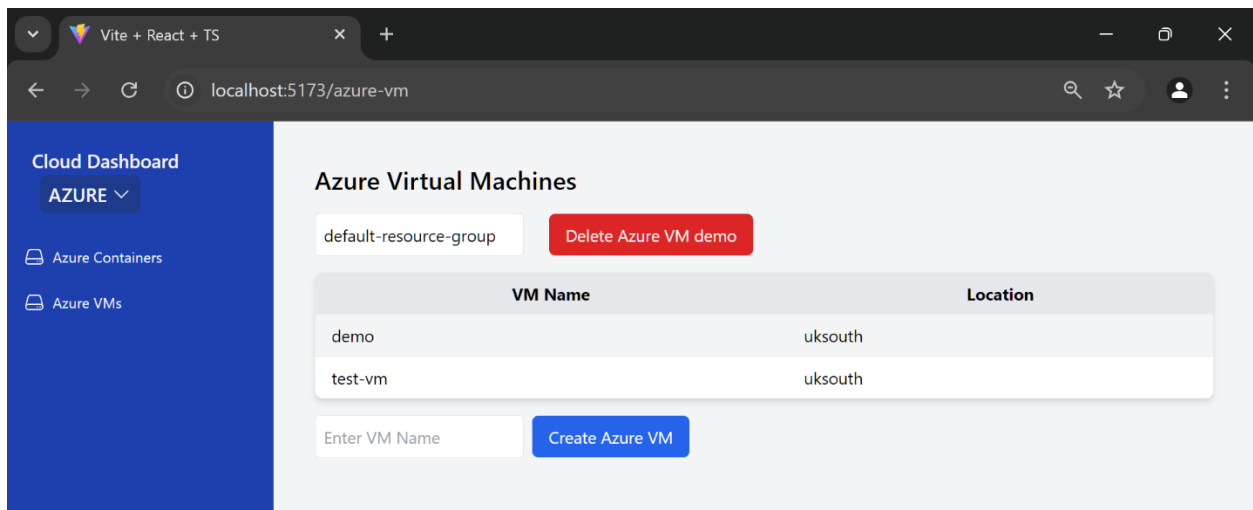


9. Screenshot of CORS implementation

```
# Allow requests from your frontend (React) URL
origins = ["*"]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

10. Screenshot of AzureVM page



11. AzureVM Component Code

```
12. import React, { useEffect, useState } from 'react';
13. import axios from 'axios';
14.
```

```

15. interface AzureVM {
16.   vm_name: string;
17.   location: string;
18. }
19.
20. const AzureVM: React.FC = () => {
21.   const [vms, setVMs] = useState<AzureVM[]>([]);
22.   const [loading, setLoading] = useState<boolean>(true);
23.   const [error, setError] = useState<string | null>(null);
24.   const [creatingVM, setCreatingVM] = useState<boolean>(false);
25.   const [isDeleting, setIsDeleting] = useState<boolean>(false);
26.   const [selectedVM, setSelectedVM] = useState<AzureVM | null>(null);
27.   const [resourceGroupName, setResourceGroupName] =
     useState<string>('default-resource-group');
28.   const [vmName, setVMName] = useState<string>('');
29.
30.   // Fetch Azure VMs
31.   const fetchAzureVMs = async () => {
32.     setLoading(true); // Start loading
33.     try {
34.       const response = await
         axios.get(`http://localhost:8000/azure/vms/multicloud-management-
           dashboard-resource-group`);
35.       setVMs(response.data.vms); // Assuming 'setVMs' is the correct state
         setter for VMs
36.     } catch (error) {
37.       console.error('Error fetching Azure VMs:', error.response ||
         error.message);
38.       setError('Failed to fetch Azure VMs');
39.     } finally {
40.       setLoading(false); // Stop loading
41.     }
42.   };
43.
44.
45.   // Create a new Azure VM
46.   const createAzureVM = async () => {
47.     setCreatingVM(true); // Set creating state
48.     try {
49.       await
         axios.post(`http://localhost:8000/vm/${resourceGroupName}/${vmName}`);
50.       alert('Azure VM created successfully');
51.       fetchAzureVMs(); // Refresh VM list after creation
52.       setVMName(''); // Clear input after creating
53.     } catch (error) {

```

```

54.     console.error('Error creating Azure VM:', error);
55.     alert('Failed to create Azure VM');
56.   } finally {
57.     setCreatingVM(false); // Stop creating state
58.   }
59. };
60.
61. // Delete the selected Azure VM
62. const deleteAzureVM = async () => {
63.   if (!selectedVM) return;
64.   setIsDeleting(true); // Start deleting
65.   try {
66.     await
67.     axios.delete(`http://localhost:8000/vm/${resourceGroupName}/${selectedVM.v
68.     m_name}`);
69.     alert(`Azure VM ${selectedVM.vm_name} deleted successfully`);
70.     fetchAzureVMs(); // Refresh the VM list after deletion
71.     setSelectedVM(null); // Clear selection after deletion
72.   } catch (error) {
73.     console.error('Error deleting Azure VM:', error);
74.     alert(`Failed to delete Azure VM ${selectedVM.vm_name}`);
75.   } finally {
76.     setIsDeleting(false); // Stop deleting
77.   }
78. };
79.
80. useEffect(() => {
81.   fetchAzureVMs();
82. }, []);
83.
84. if (loading) {
85.   return <div>Loading...</div>; // Show loading state
86. }
87.
88. if (error) {
89.   return <div>Error: {error}</div>; // Show error state
90. }
91.
92. return (
93.   <div className="p-4">
94.     <h1 className="text-2xl font-semibold mb-4">Azure Virtual
95.     Machines</h1>

```



```

96.         type="text"
97.         placeholder="Enter Resource Group Name"
98.         value={resourceGroupName}
99.         onChange={(e) => setResourceGroupName(e.target.value)}
100.         className="border rounded p-2 mr-2 mb-4"
101.     />
102.
103.     {/* Delete Azure VM Button */}
104.     {selectedVM && (
105.         <button
106.             onClick={deleteAzureVM}
107.             disabled={isDeleting || creatingVM} // Disable if creating
108.             or deleting
109.             className={`ml-4 mb-4 px-4 py-2 text-white ${isDeleting ?
110.                 'bg-gray-400' : 'bg-red-600 hover:bg-red-700'} rounded-md`}
111.             >
112.                 {isDeleting ? 'Deleting...' : `Delete Azure VM
113.                 ${selectedVM.vm_name}`}
114.             </button>
115.         )}
116.
117.     {/* Display Azure VMs */}
118.     <table className="min-w-full bg-white shadow-md rounded-lg
119.         overflow-hidden">
120.         <thead className="bg-gray-200">
121.             <tr>
122.                 <th className="py-2 px-4">VM Name</th>
123.                 <th className="py-2 px-4">Location</th>
124.             </tr>
125.         </thead>
126.         <tbody>
127.             {vms && vms.length > 0 ? (
128.                 vms.map((vm) => (
129.                     <tr
130.                         key={vm.vm_name}
131.                         className={`border-b cursor-pointer ${selectedVM?.vm_name ===
132.                             vm.vm_name ? 'bg-gray-100' : ''}`}
133.                         onClick={() => setSelectedVM(vm)}
134.                     >
135.                         <td className="py-2 px-4">{vm.vm_name}</td>

```

```

136.         <td colSpan={2} className="py-2 px-4 text-center">No VMs
        found</td>
137.     </tr>
138.   )}
139.
140.   </tbody>
141. </table>
142.
143.   {/* Create Azure VM Button */}
144.   <div className="mb-4">
145.     <input
146.       type="text"
147.       placeholder="Enter VM Name"
148.       value={vmName}
149.       onChange={(e) => setVMName(e.target.value)}
150.       className="border rounded p-2 mr-2"
151.     />
152.     <button
153.       onClick={createAzureVM}
154.       disabled={creatingVM || isDeleting} // Disable if creating
        or deleting
155.       className={`px-4 py-2 mt-4 text-white ${creatingVM ? 'bg-
        gray-400' : 'bg-blue-600 hover:bg-blue-700'} rounded-md`}
156.     >
157.       {creatingVM ? 'Creating...' : 'Create Azure VM'}
158.     </button>
159.   </div>
160. </div>
161.   );
162. };
163.
164. export default AzureVM;

```

11. Sidebar Component Code

```

12. import React, { useState } from 'react';
13. import { Link } from 'react-router-dom';
14. import { HomeIcon, ServerIcon, ArchiveBoxIcon, CloudIcon, ChevronDownIcon
    } from '@heroicons/react/24/outline';
15.
16. const Sidebar = () => {
17.   const [activeProvider, setActiveProvider] = useState<string |
    null>('aws');
18.   const [dropdownOpen, setDropdownOpen] = useState(false);

```

```

19.
20.   const handleProviderChange = (provider: string) => {
21.       setActiveProvider(provider);
22.       setDropdownOpen(false);
23.   };
24.
25.   return (
26.       <aside className="w-64 bg-blue-800 text-gray-100">
27.           <div className="p-6 text-lg font-semibold">
28.               Cloud Dashboard
29.               <button
30.                   onClick={() => setDropdownOpen(!dropdownOpen)}
31.                   className="ml-2 bg-blue-900 px-2 py-1 rounded-md flex items-
center"
32.               >
33.                   {activeProvider.toUpperCase()}
34.                   <ChevronDownIcon className="h-5 w-5 ml-1" />
35.               </button>
36.               {dropdownOpen && (
37.                   <div className="absolute bg-blue-900 p-2 mt-2 rounded-md shadow-
lg">
38.                       <ul className="space-y-1">
39.                           <li>
40.                               <button
41.                                   onClick={() => handleProviderChange('aws')}
42.                                   className="block px-4 py-2 hover:bg-blue-700 w-full
text-left"
43.                               >
44.                                   AWS
45.                               </button>
46.                           </li>
47.                           <li>
48.                               <button
49.                                   onClick={() => handleProviderChange('azure')}
50.                                   className="block px-4 py-2 hover:bg-blue-700 w-full
text-left"
51.                               >
52.                                   Azure
53.                               </button>
54.                           </li>
55.                           <li>
56.                               <button
57.                                   onClick={() => handleProviderChange('openstack')}
58.                                   className="block px-4 py-2 hover:bg-blue-700 w-full
text-left"

```

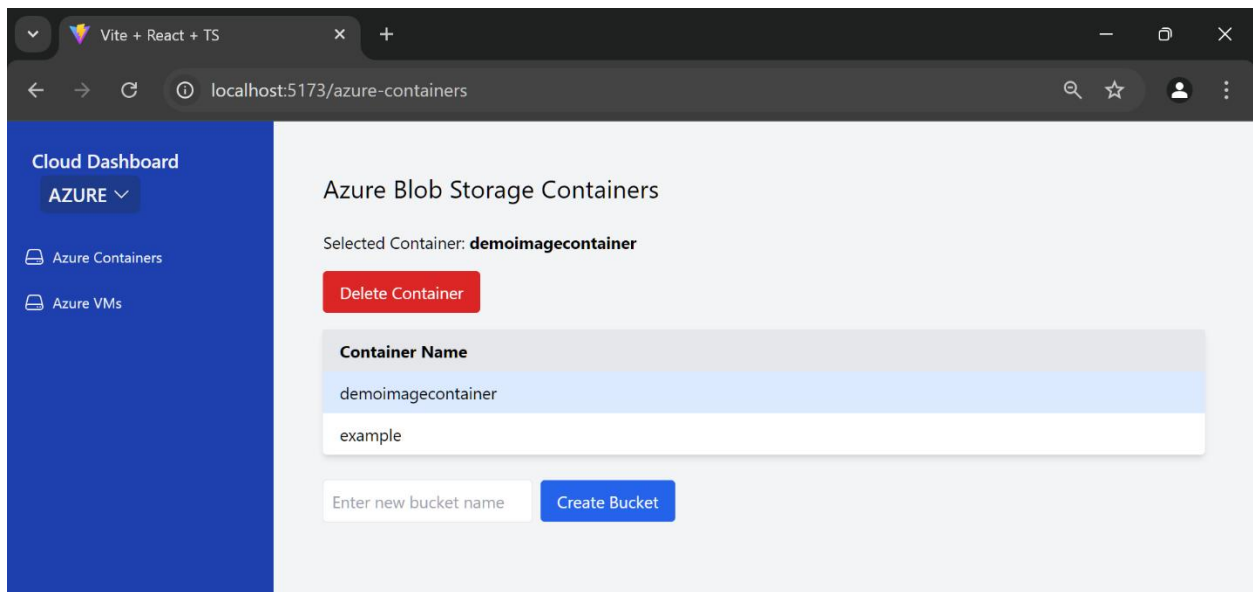
```

59.         >
60.             OpenStack
61.         </button>
62.     </li>
63. </ul>
64. </div>
65. })
66. </div>
67. <ul className="space-y-2 px-2">
68.     {activeProvider === 'aws' && (
69.         <>
70.             <li>
71.                 <Link to="/aws-ec2" className="flex items-center p-2 text-sm
72.                 hover:bg-blue-700 rounded-md">
73.                     <HomeIcon className="h-5 w-5 mr-2" />
74.                     AWS EC2 Instances
75.                 </Link>
76.             </li>
77.             <li>
78.                 <Link to="/aws-s3" className="flex items-center p-2 text-sm
79.                 hover:bg-blue-700 rounded-md">
80.                     <CloudIcon className="h-5 w-5 mr-2" />
81.                     AWS S3 Buckets
82.                 </Link>
83.             </li>
84.             <li>
85.                 <Link to="/aws-volumes" className="flex items-center p-2
86.                 text-sm hover:bg-blue-700 rounded-md">
87.                     <CloudIcon className="h-5 w-5 mr-2" />
88.                     AWS Volumes
89.                 </Link>
90.             </li>
91.         </>
92.     )}
93.     {activeProvider === 'azure' && (
94.         <>
95.             <li>
96.                 <Link to="/azure-containers" className="flex items-center p-
97.                 2 text-sm hover:bg-blue-700 rounded-md">
98.                     <ServerIcon className="h-5 w-5 mr-2" />
99.                     Azure Containers
100.                 </Link>
101.             </li>
102.             <li>

```

```
100.         <Link to="/azure-vm" className="flex items-center p-2
text-sm hover:bg-blue-700 rounded-md">
101.             <ServerIcon className="h-5 w-5 mr-2" />
102.             Azure VMs
103.         </Link>
104.     </li>
105. </>
106. }}
107. {activeProvider === 'openstack' && (
108.     <>
109.         <li>
110.             <Link to="/openstack-servers" className="flex items-
center p-2 text-sm hover:bg-blue-700 rounded-md">
111.                 <ArchiveBoxIcon className="h-5 w-5 mr-2" />
112.                 OpenStack Servers
113.             </Link>
114.         </li>
115.     </>
116. )}
117. </ul>
118. </aside>
119. );
120. };
121.
122. export default Sidebar;
```

13. Screenshot of Azure Containers Page



14. AzureContainers Component Code

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const AzureContainers = () => {
  const [containers, setContainers] = useState<string[]>([]);
  const [selectedContainer, setSelectedContainer] = useState<string | null>(null);
  const [containerName, setContainerName] = useState('');
  const [errorMessage, setErrorMessage] = useState('');
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [isCreating, setIsCreating] = useState<boolean>(false);
  const [isDeleting, setIsDeleting] = useState<boolean>(false);

  // Fetch all containers
  const fetchContainers = async () => {
    setIsLoading(true);
    try {
      const response = await axios.get('http://localhost:8000/azure/containers');
      setContainers(response.data.containers);
    } catch (error) {
      setErrorMessage('Error fetching containers');
    }
  }
}
```

```

        console.error('Error fetching containers:', error);
    } finally {
        setIsLoading(false);
    }
};

const createContainer = async () => {
    try {
        setErrorMessage(''); // Clear any previous error
        setIsLoading(true); // Set loading state
        const response = await
axios.post(`http://localhost:8000/azure/container/${containerName}`);
        alert('Container created successfully');
        console.log('Container Created:', response.data);
        setContainerName(''); // Clear input after successful creation
        fetchContainers(); // Refresh the bucket list after creation
    } catch (error: any) {
        if (error.response && error.response.data && error.response.data.detail) {
            setErrorMessage(error.response.data.detail); // Set the error message
returned from the backend
        } else {
            setErrorMessage('An unexpected error occurred. Please try again.');
```

```

// Fetch containers on component mount
useEffect(() => {
  fetchContainers();
}, []);

return (
  <div className="p-6">
    <h1 className="text-2xl mb-4">Azure Blob Storage Containers</h1>

    {/* Delete button for selected container */}
    {selectedContainer && (
      <div className="mt-6">
        <p>
          Selected Container: <strong>{selectedContainer}</strong>
        </p>
        <button
          onClick={() => deleteContainer(selectedContainer)}
          disabled={isDeleting}
          className={`bg-red-600 text-white px-4 py-2 mb-4 rounded ${
            isDeleting ? 'bg-gray-400' : 'hover:bg-red-700'
          } mt-4`}
        >
          {isDeleting ? 'Deleting...' : 'Delete Container'}
        </button>
      </div>
    )}

    {/* Display containers in a table */}
    {isLoading ? (
      <p>Loading containers...</p>
    ) : containers.length > 0 ? (
      <table className="min-w-full bg-white shadow-md rounded mb-6">
        <thead>
          <tr>
            <th className="py-2 px-4 bg-gray-200 text-left">Container Name</th>
          </tr>
        </thead>
        <tbody>
          {containers.map((container) => (
            <tr
              key={container}
              onClick={() => setSelectedContainer(container)}
              className={`cursor-pointer ${
                selectedContainer === container ? 'bg-blue-100' : ''
              }}
            >

```



```

        }`}
      >
      <td className="py-2 px-4">{container}</td>
    </tr>
  )})
</tbody>
</table>
) : (
  <p>No containers available.</p>
)}

{/* Create new container */}
<div className="mb-6">
  <input
    type="text"
    placeholder="Enter new bucket name"
    value={containerName}
    onChange={(e) => setContainerName(e.target.value)}
    className="border rounded p-2 mr-2"
    disabled={isLoading || isDeleting} // Disable input if
creating/deleting
  />
  <button
    onClick={createContainer}
    className={`bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700
${isLoading ? 'opacity-50 cursor-not-allowed' : ''}`}
    disabled={isLoading || isDeleting} // Disable button if
creating/deleting
  >
    {isLoading ? 'Creating...' : 'Create Bucket'}
  </button>
</div>

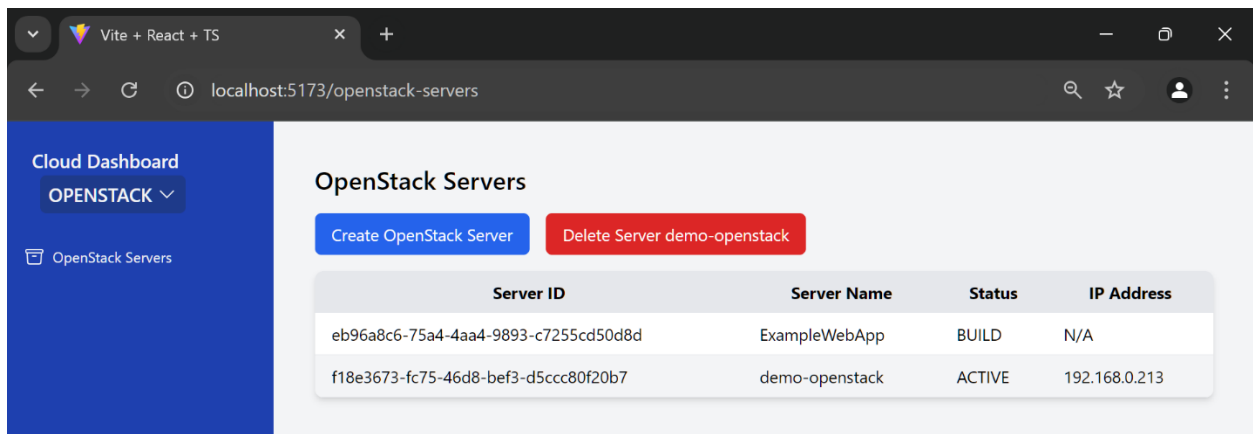
  {/* Error message */}
  {errorMessage && <div className="text-red-500 mb-4">{errorMessage}</div>}

</div>
);
};

export default AzureContainers;

```

15. Screenshot of OpenStack Servers page



16. OpenStackServers Component Code

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

interface OpenStackServer {
  id: string;
  name: string;
  status: string;
  addresses: { [key: string]: Array<{ addr: string }> };
}

const OpenStackServers: React.FC = () => {
  const [servers, setServers] = useState<OpenStackServer[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
  const [creatingServer, setCreatingServer] = useState<boolean>(false);
  const [isDeleting, setIsDeleting] = useState<boolean>(false);
  const [selectedServer, setSelectedServer] = useState<OpenStackServer | null>(null);

  // Fetch OpenStack Servers
  const fetchServers = async () => {
```

```

    setLoading(true); // Start loading
    try {
        const response = await
axios.get('http://localhost:8000/openstack/servers');
        setServers(response.data);
    } catch (error) {
        console.error('Error fetching OpenStack servers:', error);
        setError('Failed to fetch OpenStack servers');
    } finally {
        setLoading(false); // Stop loading
    }
};

// Create a new OpenStack Server
const createServer = async () => {
    setCreatingServer(true); // Set creating state
    try {
        await axios.post('http://localhost:8000/openstack/server'); // Adjust this
API call accordingly
        alert('OpenStack Server created successfully');
        fetchServers(); // Refresh server list after creation
    } catch (error) {
        console.error('Error creating OpenStack server:', error);
        alert('Failed to create OpenStack server');
    } finally {
        setCreatingServer(false); // Stop creating state
    }
};

// Delete the selected OpenStack server
const deleteServer = async () => {
    if (!selectedServer) return;
    setIsDeleting(true); // Start deleting
    try {
        await
axios.delete(`http://localhost:8000/openstack/server/${selectedServer.id}`);
        alert(`OpenStack Server ${selectedServer.id} deleted successfully`);
        fetchServers(); // Refresh the server list after deletion
        setSelectedServer(null); // Clear the selection after deletion
    } catch (error) {
        console.error('Error deleting OpenStack server:', error);
        alert(`Failed to delete OpenStack Server ${selectedServer.id}`);
    } finally {
        setIsDeleting(false); // Stop deleting
    }
}

```

```

};

useEffect(() => {
  fetchServers();
}, []);

if (loading) {
  return <div>Loading...</div>; // Show loading state
}

if (error) {
  return <div>Error: {error}</div>; // Show error state
}

return (
  <div className="p-4">
    <h1 className="text-2xl font-semibold mb-4">OpenStack Servers</h1>

    {/* Create Server Button */}
    <button
      onClick={createServer}
      disabled={creatingServer || isDeleting} // Disable if creating or
deleting
      className={`mb-4 px-4 py-2 text-white ${creatingServer ? 'bg-gray-400' :
'bg-blue-600 hover:bg-blue-700'} rounded-md`}
    >
      {creatingServer ? 'Creating...' : 'Create OpenStack Server'}
    </button>

    {/* Delete Server Button */}
    {selectedServer && (
      <button
        onClick={deleteServer}
        disabled={isDeleting || creatingServer} // Disable if creating or
deleting
        className={`ml-4 mb-4 px-4 py-2 text-white ${isDeleting ? 'bg-gray-400'
: 'bg-red-600 hover:bg-red-700'} rounded-md`}
      >
        {isDeleting ? 'Deleting...' : `Delete Server ${selectedServer.name}`}
      </button>
    )}
  </div>

  <table className="min-w-full bg-white shadow-md rounded-lg overflow-
hidden">
    <thead className="bg-gray-200">

```

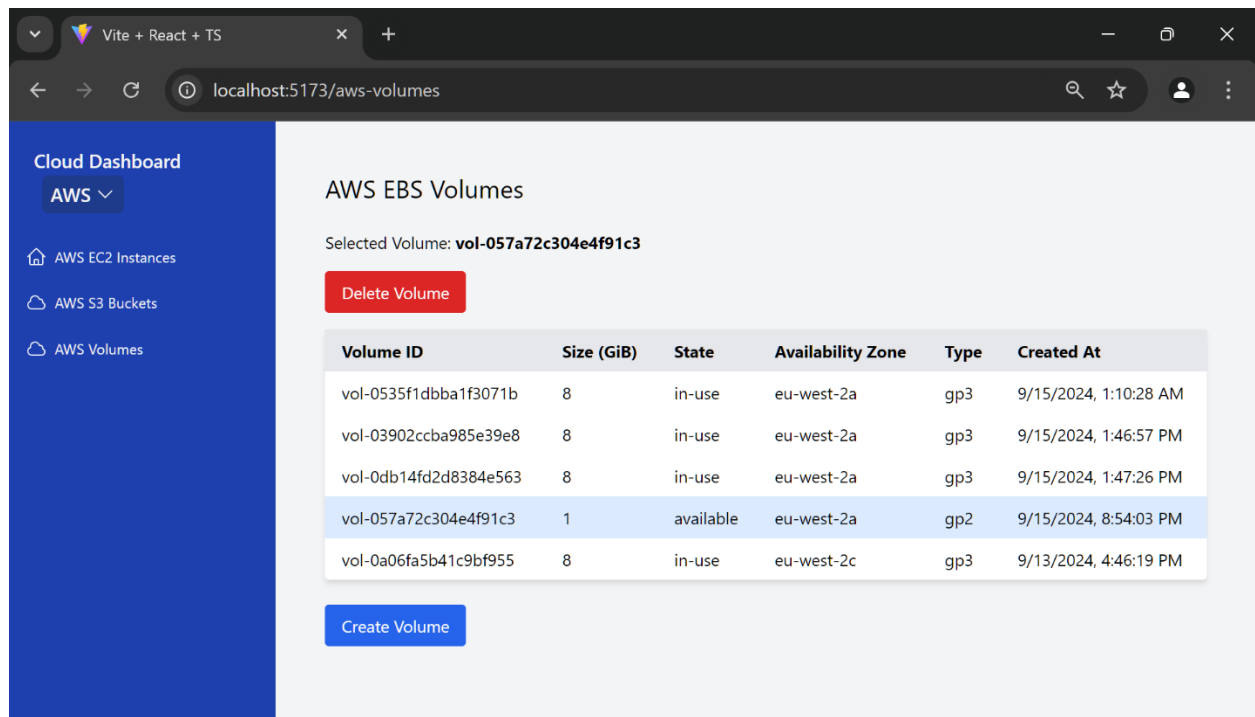
```

    <tr>
      <th className="py-2 px-4">Server ID</th>
      <th className="py-2 px-4">Server Name</th>
      <th className="py-2 px-4">Status</th>
      <th className="py-2 px-4">IP Address</th>
    </tr>
  </thead>
  <tbody>
    {servers.map((server) => (
      <tr
        key={server.id}
        className={`border-b cursor-pointer ${selectedServer?.id ===
server.id ? 'bg-gray-100' : ''}`}
        onClick={() => setSelectedServer(server)}
      >
        <td className="py-2 px-4">{server.id}</td>
        <td className="py-2 px-4">{server.name}</td>
        <td className="py-2 px-4">{server.status}</td>
        <td className="py-2 px-4">
          {Object.keys(server.addresses).length > 0
            ? server.addresses[Object.keys(server.addresses)[0]][0].addr
            : 'N/A'}
        </td>
      </tr>
    )
  )}
  </tbody>
</table>
</div>
);
};

export default OpenStackServers;

```

17. Screenshot of AWSVolumes Page



18. AWSVolumes Component Code

```
import { useState, useEffect } from 'react';
import axios from 'axios';

const AWSVolumes = () => {
  const [volumes, setVolumes] = useState<any[]>([]);
  const [selectedVolume, setSelectedVolume] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const [isDeleting, setIsDeleting] = useState<boolean>(false);

  // Fetch all volumes
  const fetchVolumes = async () => {
    try {
      const response = await axios.get('http://localhost:8000/aws/volumes');
      console.log('Volumes API Response:', response.data);
      setVolumes(response.data.volumes);
    } catch (error) {
      console.error('Error fetching volumes:', error);
    }
  };

  // Create a new volume with default parameters
```

```

const createVolume = async () => {
  try {
    setIsLoading(true); // Show loading state
    const response = await axios.post('http://localhost:8000/aws/volume');
    alert('Volume created successfully');
    console.log('Volume Created:', response.data);
    setIsLoading(false);
    fetchVolumes(); // Refresh the volume list after creation
  } catch (error) {
    console.error('Error creating volume:', error);
    setIsLoading(false);
  }
};

// Delete a volume
const deleteVolume = async (volumeId: string) => {
  try {
    setIsDeleting(true); // Show loading state
    await axios.delete(`http://localhost:8000/aws/volume/${volumeId}`);
    alert('Volume deleted successfully');
    console.log(`Volume ${volumeId} deleted`);
    setSelectedVolume(null); // Reset the selected volume
    setIsDeleting(false);
    fetchVolumes(); // Refresh the volume list after deletion
  } catch (error) {
    console.error(`Error deleting volume ${volumeId}:`, error);
    setIsDeleting(false);
  }
};

useEffect(() => {
  fetchVolumes();
}, []);

return (
  <div className="p-6">
    <h1 className="text-2xl mb-4">AWS EBS Volumes</h1>

    {/* Delete button for selected volume */}
    {selectedVolume && (
      <div className="mt-6">
        <p>
          Selected Volume: <strong>{selectedVolume}</strong>
        </p>
        <button

```

```

        onClick={() => deleteVolume(selectedVolume)}
        className="bg-red-600 text-white px-4 py-2 rounded hover:bg-red-700
mt-4 mb-4"
        disabled={isLoading}
      >
        {isDeleting ? 'Deleting...' : 'Delete Volume'}
      </button>
    </div>
  )}

  { /* Table for displaying volumes */ }
  {Array.isArray(volumes) && volumes.length > 0 ? (
    <table className="min-w-full bg-white shadow-md rounded mb-6">
      <thead>
        <tr>
          <th className="py-2 px-4 bg-gray-200 text-left">Volume ID</th>
          <th className="py-2 px-4 bg-gray-200 text-left">Size (GiB)</th>
          <th className="py-2 px-4 bg-gray-200 text-left">State</th>
          <th className="py-2 px-4 bg-gray-200 text-left">Availability
Zone</th>
          <th className="py-2 px-4 bg-gray-200 text-left">Type</th>
          <th className="py-2 px-4 bg-gray-200 text-left">Created At</th>
        </tr>
      </thead>
      <tbody>
        {volumes.map((volume: any) => (
          <tr
            key={volume.VolumeId}
            onClick={() => setSelectedVolume(volume.VolumeId)}
            className={`cursor-pointer ${selectedVolume === volume.VolumeId ?
'bg-blue-100' : ''}`}
          >
            <td className="py-2 px-4">{volume.VolumeId}</td>
            <td className="py-2 px-4">{volume.Size}</td>
            <td className="py-2 px-4">{volume.State}</td>
            <td className="py-2 px-4">{volume.AvailabilityZone}</td>
            <td className="py-2 px-4">{volume.VolumeType}</td>
            <td className="py-2 px-4">{new
Date(volume.CreateTime).toLocaleString()}</td>
          </tr>
        )
        )}
      </tbody>
    </table>
  ) : (
    <p>No volumes available.</p>

```



```

    })

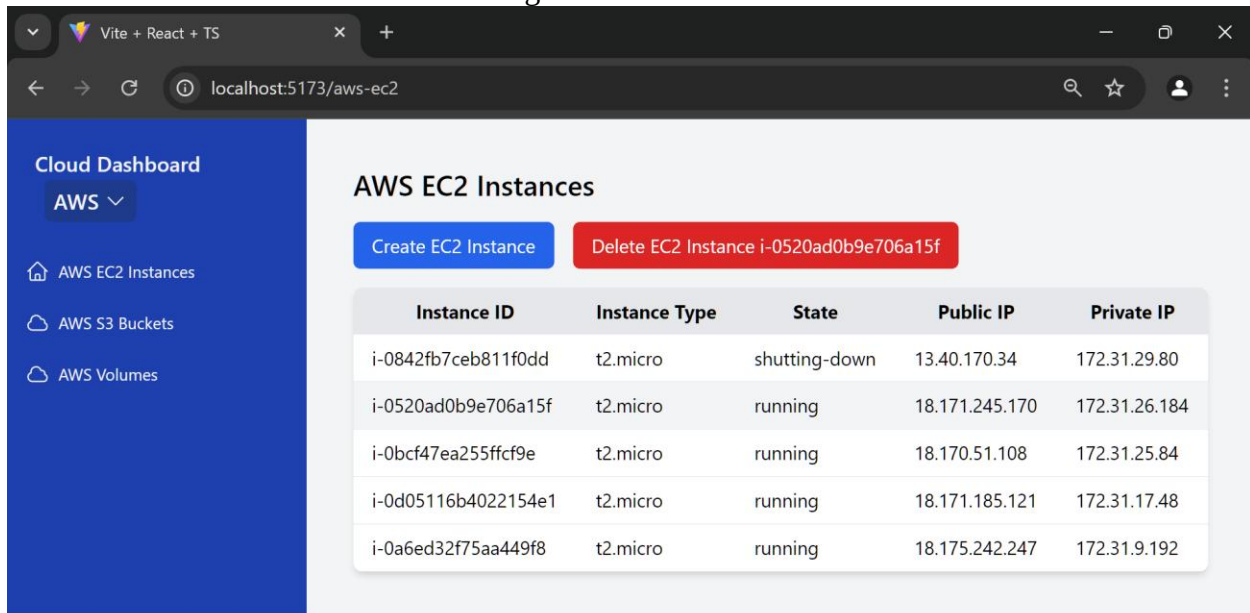
    /* Create new volume button */
    <div className="mb-6">
      <button
        onClick={createVolume}
        className={`bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700
        ${isLoading ? 'opacity-50 cursor-not-allowed' : ''}`}
        disabled={isLoading}
      >
        {isLoading ? 'Creating...' : 'Create Volume'}
      </button>
    </div>

  </div>
);
};

export default AWSVolumes;

```

19. Screenshot of AWSEC2Instances Page



The screenshot shows the AWS Cloud Dashboard interface. On the left is a blue sidebar with the text 'Cloud Dashboard' and 'AWS' selected. Below this are links for 'AWS EC2 Instances', 'AWS S3 Buckets', and 'AWS Volumes'. The main content area is titled 'AWS EC2 Instances'. At the top of this area are two buttons: 'Create EC2 Instance' (blue) and 'Delete EC2 Instance i-0520ad0b9e706a15f' (red). Below these buttons is a table with the following data:

Instance ID	Instance Type	State	Public IP	Private IP
i-0842fb7ceb811f0dd	t2.micro	shutting-down	13.40.170.34	172.31.29.80
i-0520ad0b9e706a15f	t2.micro	running	18.171.245.170	172.31.26.184
i-0bcf47ea255ffc9e	t2.micro	running	18.170.51.108	172.31.25.84
i-0d05116b4022154e1	t2.micro	running	18.171.185.121	172.31.17.48
i-0a6ed32f75aa449f8	t2.micro	running	18.175.242.247	172.31.9.192

20. AWSEC2Instances Component Code

```
import React, { useEffect, useState } from 'react';
```

```

import axios from 'axios';

interface EC2Instance {
  InstanceId: string;
  InstanceType: string;
  State: { Name: string };
  PublicIpAddress?: string;
  PrivateIpAddress: string;
}

const AWSEC2Instances: React.FC = () => {
  const [instances, setInstances] = useState<EC2Instance[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
  const [creatingInstance, setCreatingInstance] = useState<boolean>(false);
  const [isDeleting, setIsDeleting] = useState<boolean>(false);
  const [selectedInstance, setSelectedInstance] = useState<EC2Instance |
null>(null);

  // Fetch EC2 Instances
  const fetchEC2Instances = async () => {
    setLoading(true); // Start loading
    try {
      const response = await axios.get('http://localhost:8000/aws/instances');
      const reservations = response.data.ec2_instances.Reservations;
      const allInstances = reservations.flatMap((reservation: any) =>
reservation.Instances);
      setInstances(allInstances);
    } catch (error) {
      setError('Failed to fetch EC2 instances');
    } finally {
      setLoading(false); // Stop loading
    }
  };

  // Create a new EC2 instance
  const createEC2Instance = async () => {
    setCreatingInstance(true); // Set creating state
    try {
      await axios.post('http://localhost:8000/aws/instance');
      alert('EC2 Instance created successfully');
      fetchEC2Instances(); // Refresh instance list after creation
    } catch (error) {
      console.error('Error creating EC2 instance:', error);
      alert('Failed to create EC2 instance');
    }
  };

```

```

    } finally {
      setCreatingInstance(false); // Stop creating state
    }
  };

  // Delete the selected EC2 instance
  const deleteEC2Instance = async () => {
    if (!selectedInstance) return;
    setIsDeleting(true); // Start deleting
    try {
      await
axios.delete(`http://localhost:8000/aws/instance/${selectedInstance.InstanceId}`)
;
      alert(`EC2 Instance ${selectedInstance.InstanceId} deleted successfully`);
      fetchEC2Instances(); // Refresh the instance list after deletion
      setSelectedInstance(null); // Clear the selection after deletion
    } catch (error) {
      console.error('Error deleting EC2 instance:', error);
      alert(`Failed to delete EC2 Instance ${selectedInstance.InstanceId}`);
    } finally {
      setIsDeleting(false); // Stop deleting
    }
  };

  useEffect(() => {
    fetchEC2Instances();
  }, []);

  if (loading) {
    return <div>Loading...</div>; // Show loading state
  }

  if (error) {
    return <div>Error: {error}</div>; // Show error state
  }

  return (
    <div className="p-4">
      <h1 className="text-2xl font-semibold mb-4">AWS EC2 Instances</h1>

      { /* Create EC2 Instance Button */ }
      <button
        onClick={createEC2Instance}
        disabled={creatingInstance || isDeleting} // Disable if creating or
deleting

```

```

        className={`mb-4 px-4 py-2 text-white ${creatingInstance ? 'bg-gray-400'
: 'bg-blue-600 hover:bg-blue-700'} rounded-md`}
      >
        {creatingInstance ? 'Creating...' : 'Create EC2 Instance'}
      </button>

      {/* Delete EC2 Instance Button */}
      {selectedInstance && (
        <button
          onClick={deleteEC2Instance}
          disabled={isDeleting || creatingInstance} // Disable if creating or
deleting
          className={`ml-4 mb-4 px-4 py-2 text-white ${isDeleting ? 'bg-gray-400'
: 'bg-red-600 hover:bg-red-700'} rounded-md`}
        >
          {isDeleting ? 'Deleting...' : `Delete EC2 Instance
${selectedInstance.InstanceId}`}
        </button>
      )}

      <table className="min-w-full bg-white shadow-md rounded-lg overflow-
hidden">
        <thead className="bg-gray-200">
          <tr>
            <th className="py-2 px-4">Instance ID</th>
            <th className="py-2 px-4">Instance Type</th>
            <th className="py-2 px-4">State</th>
            <th className="py-2 px-4">Public IP</th>
            <th className="py-2 px-4">Private IP</th>
          </tr>
        </thead>
        <tbody>
          {instances.map((instance) => (
            <tr
              key={instance.InstanceId}
              className={`border-b cursor-pointer ${selectedInstance?.InstanceId
=== instance.InstanceId ? 'bg-gray-100' : ''}`}
              onClick={() => setSelectedInstance(instance)}
            >
              <td className="py-2 px-4">{instance.InstanceId}</td>
              <td className="py-2 px-4">{instance.InstanceType}</td>
              <td className="py-2 px-4">{instance.State.Name}</td>
              <td className="py-2 px-4">{instance.PublicIpAddress || 'N/A'}</td>
              <td className="py-2 px-4">{instance.PrivateIpAddress}</td>
            </tr>
          ))}
        </tbody>
      </table>

```

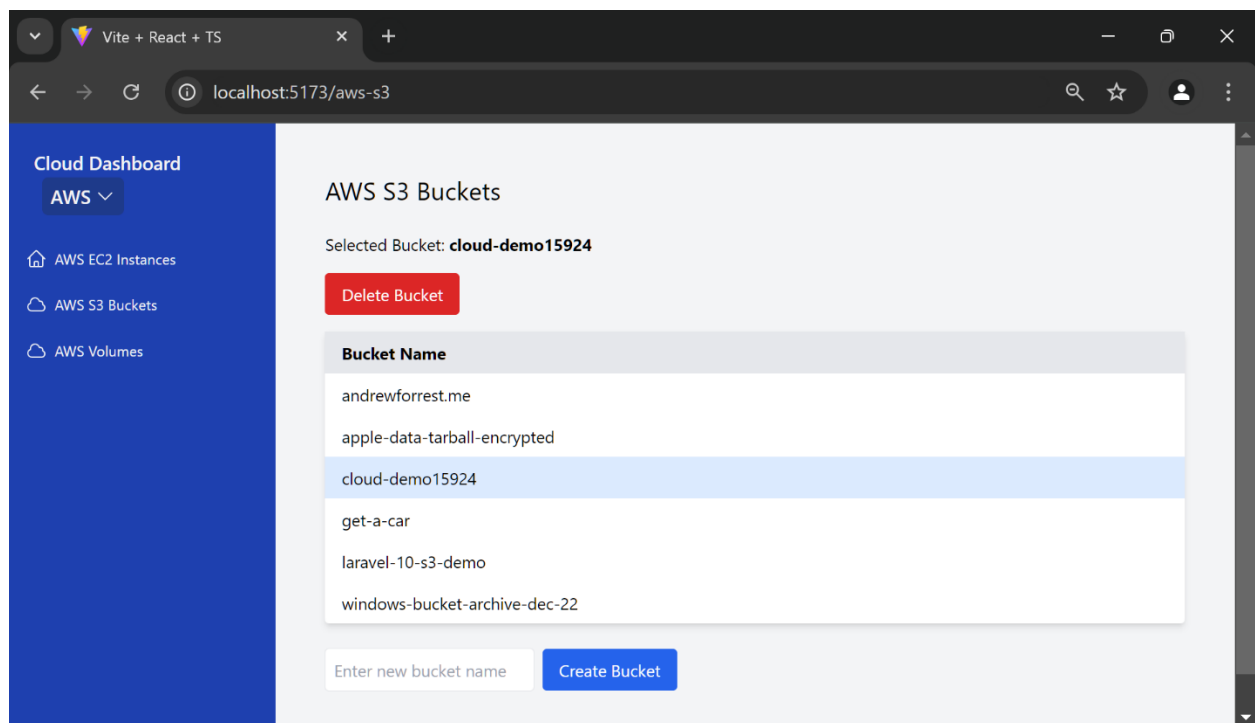
```

    )))}
  </tbody>
</table>
</div>
);
};

export default AWSEC2Instances;

```

21. Screenshot of S3Buckets Page



22. S3Buckets Component Code

```

import { useState, useEffect } from 'react';
import axios from 'axios';

const S3Buckets = () => {
  const [buckets, setBuckets] = useState<any[]>([]); // Initialize as an empty array
  const [selectedBucket, setSelectedBucket] = useState<string | null>(null);
  const [bucketName, setBucketName] = useState('');
  const [errorMessage, setErrorMessage] = useState(''); // For error messages

```

```

    const [isLoading, setIsLoading] = useState<boolean>(false); // Loading state
    for bucket creation
    const [isDeleting, setIsDeleting] = useState<boolean>(false); // Loading state
    for bucket deletion

    // Fetch all S3 buckets
    const fetchBuckets = async () => {
        try {
            const response = await axios.get('http://localhost:8000/aws/buckets');
            console.log('S3 Buckets API Response:', response.data); // Log the response
            to inspect it
            setBuckets(response.data.response.Buckets); // Update to match the nested
            structure
        } catch (error) {
            console.error('Error fetching S3 buckets:', error);
        }
    };

    // Create a new S3 bucket
    const createBucket = async () => {
        try {
            setErrorMessage(''); // Clear any previous error
            setIsLoading(true); // Set loading state
            const response = await
            axios.post(`http://localhost:8000/aws/bucket/${bucketName}`);
            alert('Bucket created successfully');
            console.log('S3 Bucket Created:', response.data);
            setBucketName(''); // Clear input after successful creation
            fetchBuckets(); // Refresh the bucket list after creation
        } catch (error: any) {
            if (error.response && error.response.data && error.response.data.detail) {
                setErrorMessage(error.response.data.detail); // Set the error message
                returned from the backend
            } else {
                setErrorMessage('An unexpected error occurred. Please try again.');
```

```

    setIsDeleting(true); // Set deleting state
    await axios.delete(`http://localhost:8000/aws/bucket/${bucketName}`);
    alert('Bucket deleted successfully');
    fetchBuckets(); // Refresh buckets after deletion
    setSelectedBucket(null); // Reset selected bucket
  } catch (error) {
    console.error('Error deleting S3 bucket:', error);
  } finally {
    setIsDeleting(false); // Stop deleting state
  }
};

useEffect(() => {
  fetchBuckets();
}, []);

return (
  <div className="p-6">
    <h1 className="text-2xl mb-4">AWS S3 Buckets</h1>

    {/* Delete button for selected bucket */}
    {selectedBucket && (
      <div className="mt-6">
        <p>
          Selected Bucket: <strong>{selectedBucket}</strong>
        </p>
        <button
          onClick={() => deleteBucket(selectedBucket)}
          className={`bg-red-600 text-white px-4 py-2 rounded hover:bg-red-700
mt-4 mb-4 ${isDeleting ? 'opacity-50 cursor-not-allowed' : ''}`}
          disabled={isDeleting || isLoading} // Disable if creating/deleting
        >
          {isDeleting ? 'Deleting...' : 'Delete Bucket'}
        </button>
      </div>
    )}

    {/* Table for displaying S3 buckets */}
    {Array.isArray(buckets) && buckets.length > 0 ? (
      <table className="min-w-full bg-white shadow-md rounded mb-6">
        <thead>
          <tr>
            <th className="py-2 px-4 bg-gray-200 text-left">Bucket Name</th>
          </tr>
        </thead>

```

```

    <tbody>
      {buckets.map((bucket: { Name: string }) => (
        <tr
          key={bucket.Name}
          onClick={() => setSelectedBucket(bucket.Name)}
          className={`cursor-pointer ${selectedBucket === bucket.Name ?
'bg-blue-100' : ''}`}
        >
          <td className="py-2 px-4">{bucket.Name}</td>
        </tr>
      ))}
    </tbody>
  </table>
) : (
  <p>No S3 buckets available.</p>
)}

{/* Create new S3 bucket */}
<div className="mb-6">
  <input
    type="text"
    placeholder="Enter new bucket name"
    value={bucketName}
    onChange={(e) => setBucketName(e.target.value)}
    className="border rounded p-2 mr-2"
    disabled={isLoading || isDeleting} // Disable input if
creating/deleting
  />
  <button
    onClick={createBucket}
    className={`bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700
${isLoading ? 'opacity-50 cursor-not-allowed' : ''}`}
    disabled={isLoading || isDeleting} // Disable button if
creating/deleting
  >
    {isLoading ? 'Creating...' : 'Create Bucket'}
  </button>
</div>

{/* Error message for bucket creation */}
{errorMessage && (
  <div className="text-red-500 mb-4">
    {errorMessage}
  </div>
)}

```



```
    </div>  
  );  
};  
  
export default S3Buckets;
```